QUESTÕES DE REVISÃO

Q1 - Implemente uma função chamada criar_matriz_invertida.

A função deve receber como parâmetros:
- um ponteiro para ponteiro de inteiros, representando a matriz de entrada;
- um inteiro linhas;
- um inteiro colunas.

A função deve retornar o endereço principal da nova matriz criada, ou seja, o mesmo endereço que representa a matriz de saída.

A função deve alocar dinamicamente uma nova matriz com a mesma quantidade de linhas e colunas da matriz de entrada.

Ao percorrer a matriz de saída da esquerda para a direita e de cima para baixo, copie os elementos da matriz de entrada na ordem inversa.

Isso significa que:
- a primeira posição da matriz de saída deve receber o último elemento da matriz de entrada;
- a segunda posição da matriz de saída deve receber o penúltimo elemento da matriz de entrada;
- e assim por diante, até preencher toda a matriz.

Exemplo:
linhas = 2
colunas = 4
matriz =
1 2 3 4
5 6 7 8

saída esperada:
8 7 6 5
4 3 2 1

Retorno esperado da função:
Endereço principal da matriz de saída criada.

Implemente uma função main() que:
(1) defina os inteiros linhas e colunas;
(2) aloque dinamicamente uma matriz de entrada já preenchida com valores quaisquer;
(3) chame a função criar_matriz_invertida passando a matriz, linhas e colunas como parâmetros;
(4) receba o ponteiro retornado;

(5) imprima a matriz resultante.

Q2 - Defina uma struct chamada Personagem contendo:
- um ponteiro para char chamado nome;
- um inteiro vida;
- um inteiro ataque;
- um inteiro defesa;
- um inteiro quantidade_itens;
- um ponteiro para ponteiro de char chamado itens;
- um ponteiro para inteiro chamado quantidades_itens.

Implemente uma função chamada substituir_personagem.

A função deve receber como parâmetros:
- um ponteiro para vetor de structs do tipo Personagem, representando a entrada;
- um ponteiro para vetor de structs do tipo Personagem, representando a saída;
- um inteiro tamanho;
- um inteiro posicao, indicando em qual posição deve ocorrer a substituição;
- um ponteiro para char, representando o nome do novo personagem;
- um inteiro vida;
- um inteiro ataque;
- um inteiro defesa;
- um ponteiro para vetor de ponteiros para char, representando os itens do novo personagem;
- um ponteiro para vetor de inteiros, representando a quantidade de cada item do novo personagem;
- um inteiro quantidade_itens.

A função deve retornar o endereço de memória da posição correspondente no vetor de saída.

A função não deve alocar dinamicamente nenhuma struct, pois os vetores de entrada e de saída já existirão na função main().
Considere sempre que 0 <= posicao < tamanho.

A função deve copiar todos os personagens do vetor de entrada para o vetor de saída.
Na posição indicada por posicao, a função deve substituir o personagem original pelos dados do novo personagem informados nos parâmetros.

Para cada posição do vetor de saída, a função deve alocar dinamicamente:
- uma cópia da string do nome;
- um vetor de strings para os itens;
- uma cópia de cada string de item;
- um vetor de inteiros para armazenar a quantidade de cada item.

Os campos vida, ataque e defesa devem ser copiados do vetor de entrada, exceto na posição substituída, que deve receber os valores do novo personagem.
Os itens e as quantidades também devem ser copiados para o vetor de saída nas posições correspondentes.
Todas as cópias no vetor de saída devem ser independentes do vetor de entrada.
Não é permitido apenas copiar os ponteiros do vetor de entrada para o vetor de saída.

Para esta questão:
- se um personagem tiver pelo menos um item, use calloc para criar inicialmente os vetores itens e quantidades_itens com espaço para 1 posição;
- use realloc para aumentar os vetores itens e quantidades_itens de cada personagem à medida que cada novo item for sendo copiado;
- use calloc para criar a string do nome e cada string de item.

Em cada personagem do vetor de saída, o campo quantidade_itens deve indicar quantos itens foram armazenados.
Se um personagem tiver quantidade_itens igual a 0, os campos itens e quantidades_itens devem receber NULL.

Exemplo:
ANTES DA CHAMADA:

entrada[0] = Lia
vida = 90, ataque = 20, defesa = 15
quantidade_itens = 2
itens: adaga - 1, poção - 2

entrada[1] = Breno
vida = 140, ataque = 30, defesa = 25
quantidade_itens = 1
itens: machado - 1

entrada[2] = Ciro
vida = 100, ataque = 18, defesa = 20
quantidade_itens = 2
itens: lança - 1, escudo - 1

saida[0], saida[1] e saida[2]:
vetor declarado, ainda sem dados válidos

tamanho = 3
posicao = 2

novo personagem:

nome = Aria
vida = 120
ataque = 35
defesa = 18
quantidade_itens = 3
itens: espada - 1, escudo - 1, poção - 4

DEPOIS DA CHAMADA:

saida[0] = Lia
vida = 90, ataque = 20, defesa = 15
itens: adaga - 1, poção - 2

saida[1] = Breno
vida = 140, ataque = 30, defesa = 25
itens: machado - 1

saida[2] = Aria
vida = 120, ataque = 35, defesa = 18
itens: espada - 1, escudo - 1, poção - 4

Retorno esperado da função:
Endereço de memória da posição 2 do vetor de saída, após a substituição.

Implemente uma função main() que:
(1) defina um vetor de entrada com 3 personagens já preenchidos, um vetor de saída declarado com 3 posições ainda sem dados válidos, as informações do novo personagem e um inteiro posicao;
(2) chame a função substituir_personagem passando o vetor de entrada, o vetor de saída, o tamanho, a posição e os dados do novo personagem como parâmetros;
(3) receba o ponteiro retornado;
(4) imprima os personagens do vetor de saída e destaque os dados da struct apontada pelo retorno.